KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 6 Quiz

FastAPI I: HTTP'yi Koda Dökmek

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (6 puan)

4 • Değerlendirme Hedef 1

`app/rotalar/etkinlikler.py` dosyasında `GET /etkinlikler` rotasına tanımlı bir `mekan: str | None = None` parametresi vardır; bu parametre yol şablonunda (`{}` içinde) hiç geçmez. Aynı dosyada `GET /etkinlikler/{etkinlik_id}` (`etkinlik_id: int`) rotası da tanımlıdır. `/etkinlikler?mekan=Salon+A` isteği hangi rotaya eşleşir, `mekan` parametresi FastAPI tarafından nasıl sınıflandırılır?

(a) `/etkinlikler` rotasına eşleşir; `mekan` yol şablonunda geçmediği için sorgu parametresi sayılır ve isteğe bağlıdır.

(b) `/etkinlikler/{etkinlik_id}` rotasına eşleşir; `mekan` yol parametresi sayılır.

(c) Hiçbir rotaya eşleşmez, `404` döner çünkü `mekan` tanımsız bir yol parçasıdır.

(d) İki rotaya da eşleşir; FastAPI hangisinin çağrılacağını rastgele seçer.

Soru 2 (6 puan)

4 • Değerlendirme Hedef 1

`uvicorn app.main:app --reload` komutunu çalıştırdığınızda, gerçekte TCP bağlantısını dinleyip HTTP protokolünü konuşan süreç hangisidir; FastAPI'nin (`app` nesnesinin) rolü nedir?

(a) FastAPI TCP bağlantısını dinler; `uvicorn` yalnızca `--reload` bayrağıyla dosya değişikliklerini izleyen bir yardımcı araçtır.

(b) `uvicorn` TCP bağlantısını dinleyen ASGI sunucusudur; FastAPI (`app` nesnesi) hangi yola hangi Python fonksiyonunun cevap vereceğini tanımlayan çerçevedir — HTTP'yi bizzat konuşmaz.

(c) `uvicorn` ve FastAPI aynı görevi görür; ikisi birbirinin yerine geçebilir, biri olmadan diğeri de çalışır.

(d) `uvicorn` yalnızca geliştirme ortamında kullanılır; üretimde HTTP isteklerini doğrudan FastAPI'nin kendisi dinler.

Soru 3 (8 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki kesitler `app/main.py` ve `app/rotalar/etkinlikler.py` dosyalarından alınmıştır.

```
app.include_router(etkinlikler.router)
app.include_router(koltuklar.router)
```

Soru 3 (devamı)

```
3 app.include_router(biletler.router)
```

```
1 @router.get("/{etkinlik_id}", response_model=Etkinlik)
2 def etkinlik_detay(etkinlik_id: int, depo: Depo = Depends(get_depo)) -> Etkinlik:
3     etkinlik = depo.etkinlik_getir(etkinlik_id)
4     if etkinlik is None:
5         raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
6     return etkinlik
```

(a) Sunucu yeni başlatılmış, depo henüz boşken `GET /etkinlikler/7` isteği gönderilirse hangi durum kodu döner, bunu hangi satır üretir? (b) `app.include_router(etkinlikler.router)` satırı hiç yazılmasaydı, aynı isteğe (`GET /etkinlikler/7`) ne olurdu — durum kodu aynı mı kalırdı, cevap gövdesi farklı mı olurdu?

Soru 4 (10 puan)

4 • Değerlendirme Hedef 1

`python -m venv .venv` Kurmadan, doğrudan bilgisayarınızdaki küresel (global) `pip install fastapi` ile paketleri kursaydınız hangi sorunlarla karşılaşabilirdiniz? `requirements.txt` / `requirements-dev.txt` ayrımının bu sorunla ilişkisini de, malzeme çekmecesi benzetmesini kullanarak açıklayınız.

Soru 5 (6 puan)

4 • Değerlendirme Hedef 2

`EtkinlikOlustur` modelinde `id` alanı yoktur, ama `Etkinlik` modeli (`EtkinlikOlustur`'dan türer) buna `id: int` ekler. Bunun yerine tek bir model (`id` alanı opsiyonel, varsayılan `None`) kullansaydınız ne kaybederdiniz?

(a) Hiçbir şey kaybetmezsiniz; `response_model=Etkinlik` zaten yalnızca dönüş şemasındaki alanları filtreler, giriş modelinde `id` bulunması sonucu etkilemez.

(b) Pydantic, `POST` gövdesinde gelen `id` alanını otomatik olarak reddeder; bu yüzden tek model kullanmak güvenli kalır.

(c) İstemci isteğe `id` alanı ekleyip göndermeyi deneyebilir; tek model (`id` opsiyonel) bunu engellemez — sunucunun kendi atadığı `id`'nin yerine istemcinin gönderdiği bir değer karışabilir; ayrı giriş modeli bu alanı hiç tanımlamayarak bu riski yapısal olarak ortadan kaldırır.

(d) `EtkinlikOlustur` ile `Etkinlik`'i birleştirmek `/docs` sayfasının hiç üretilememesine yol açar.

Soru 6 (6 puan)

4 • Değerlendirme Hedef 2

`KamelModel` taban sınıfında `populate_by_name=True` ayarı olmasaydı (yalnızca `alias_generator=to_camel` kalsaydı), Python içinde doğrudan snake_case anahtar kelime argümanlarıyla (`EtkinlikOlustur(baslik=..., tarih=..., mekan=..., kapasite=..., bilet_fiyati=...)`) bir nesne oluşturmaya çalışsaydınız (ör. bir testte) ne olurdu?

(a) Sorun olmaz; Pydantic modelleri varsayılan olarak hem alan adını hem alias'ı kabul eder, `populate_by_name` yalnızca fazladan bir güvenlik katmanıdır.

(b) `bilet_fiyati` anahtarı sessizce yok sayılır, alan varsayılan değerinde kalır, hiçbir hata verilmez.

(c) Model yalnızca pozisyonel argümanlarla oluşturulabilir hale gelir; anahtar kelime argümanları tamamen devre dışı kalır.

(d) Bir doğrulama hatası (`ValidationError`) fırlatılır; model alıas (camelCase) olmayan bir anahtar kelime argümanını tanımaz — `populate_by_name=True` tam olarak bu durumu (hem alan adıyla hem alias ile doldurabilme) mümkün kılar.

Soru 7 (8 puan)

4 • Değerlendirme Hedef 2

Aşağıdaki kesit `app/modeller.py` dosyasından alınmıştır.

```python
class EtkinlikOlustur(KamelModel):
    """İstemcinin gönderdiği, henüz id'si olmayan etkinlik verisi."""

    baslik: str = Field(min_length=1)
    tarih: str
    mekan: str = Field(min_length=1)
    kapasite: int = Field(gt=0)
    bilet_fiyati: float = Field(gt=0)

class Etkinlik(EtkinlikOlustur):
    """Sunucunun döndürdüğü, kimliği atanmış tam etkinlik nesnesi."""

    id: int
```

(a) İstemci `POST /etkinlikler`'e `{"baslik": "", "tarih": "2026-11-14", "mekan": "Salon", "kapasite": 50, "biletFiyati": 100}` (`baslik` boş dize) gönderirse ne olur, bunu hangi satır yakalar? (b) `class Etkinlik(EtkinlikOlustur):` satırındaki miras ilişkisi olmasaydı (iki sınıf birbirinden bağımsız, alanları tekrar yazılarak tanımlansaydı) ne kaybederdiniz?

Soru 8 (10 puan)

4 • Değerlendirme Hedef 2

`KoltukDurumu` tipi `Literal["bos", "secili", "dolu"]` biçiminde üç değerli tanımlanmıştır, ama bu haftanın sunucu mantığı (rotalar) yalnızca `'bos'` ile `'dolu'` arasında geçiş yapar — `'secili'` değeri sunucu tarafında hiç kullanılmaz. Bunun bir tutarsızlık olmadığını, Hf.3'ün TS union tipiyle ilişkisini kurarak açıklayınız.

Soru 8 (devamı)

Soru 9 (6 puan)

4 • Değerlendirme Hedef 3

`PATCH /koltuklar/{koltuk_id}` rotasının gövde tipi yalnızca `durum` alanını içeren `KoltukGuncelle`'dir. Bir istemci bu isteğe `{"sira": 99, "durum": "bos"}` gönderirse (yani `KoltukGuncelle`'de olmayan bir `sira` alanı da eklerse) ne olur?

(a) `sira` alanı sessizce yok sayılır (Pydantic varsayılan olarak modelde tanımlı olmayan fazladan alanları yok sayar); yalnızca `durum` işlenir, koltuğun `sira`'sı değişmez.

(b) `422` doğrulama hatası döner; `KoltukGuncelle` modeli tanımlı olmayan bir alanı kabul etmez.

(c) `sira` alanı da koltuğa yazılır; `KoltukGuncelle` `PATCH` isteklerinde otomatik olarak tüm koltuk alanlarını kabul edecek şekilde genişler.

(d) İstek `500 Internal Server Error` ile başarısız olur; fazladan alan sunucu tarafında bir İSTİSNAya yol açar.

Soru 10 (6 puan)

4 • Değerlendirme Hedef 3

Zaten `'dolu'` durumundaki bir koltuk için `POST /biletler` çağrıldığında `bilet_olustur` fonksiyonu `HTTPException(status_code=409, ...)` fırlatır, `HTTPException(status_code=400, ...)` değil. Bu seçimin gerekçesi nedir?

(a) `400` ve `409` birbirinin yerine geçebilir; FastAPI ikisini de aynı biçimde işler, seçim tamamen keyfidir.

(b) `400`, isteğin gövdesi (JSON biçimi/alan tipleri) hatalı olduğunda kullanılır; burada `koltuk_id` ve `kullanici_adi` biçim olarak tamamen geçerlidir — sorun isteğin kendisinde değil, sunucudaki mevcut durumla (koltuk zaten dolu) çelişmesindedir; `409` bu "geçerli istek, çelişen durum" senaryosu içindir.

(c) `409` yalnızca veritabanı hatalarında kullanılır; bu haftanın bellekte deposu bir veritabanı olmadığı için `409` teknik olarak yanlış bir seçimdir.

(d) `400`, istemcinin yetkisiz olduğu durumlar içindir; koltuk rezervasyonu bir yetkilendirme sorunu olduğu için `409` uygundur.

Soru 11 (8 puan)

4 • Değerlendirme Hedef 3

Aşağıdaki kesit `app/rotalar/biletler.py` dosyasından alınmıştır.

```python
@router.post("", response_model=Bilet, status_code=201)
def bilet_olustur(veri: BiletOlustur, depo: Depo = Depends(get_depo)) -> Bilet:
```

Soru 11 (devamı)

```
    koltuk = depo.koltuk_getir(veri.koltuk_id)
    if koltuk is None:
        raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
    if koltuk.durum == "dolu":
        raise HTTPException(status_code=409, detail="Bu koltuk zaten satılmış")
    bilet = depo.bilet_ekle(veri)
    depo.koltuk_guncelle(koltuk.id, "dolu")
    return bilet
```

(a) Aynı `koltuk_id` ile ardı ardına gönderilen iki `curl` isteğinden (birincisi `201`, ikincisi `409` döner) `depo.koltuk_guncelle` fonksiyonu toplam kaç kez çağrılır? (b) `raise HTTPException(...)` satırı çalıştığında, fonksiyonun altındaki `depo.bilet_ekle`, `depo.koltuk_guncelle` ve `return bilet` satırları çalışır mı? Bunu Python'ın istisna (*exception*) mekanizmasıyla ilişkilendirerek açıklayınız.

Soru 12 (10 puan)

4 • Değerlendirme Hedef 3

Bir `Etkinlik` silindiğinde, ona bağlı `Koltuk` ve `Bilet` kayıtları depodan neden otomatik olarak silinmez (cascade silme neden bilinçli olarak yapılmaz)? Ayrıca rotaların `APIRouter` ile üç ayrı dosyaya (`etkinlikler.py`, `koltuklar.py`, `biletler.py`) bölünmesinin, YG II'deki hangi disiplinle bağlantılı olduğunu açıklayınız.

Soru 13 (6 puan)

4 • Değerlendirme Hedef 4

`tests/conftest.py`'deki `client` fixture'ı `app.dependency_overrides[get_depo] = lambda: test_depo` satırıyla her testte yeni, boş bir `Depo()` nesnesi atar. Rota fonksiyonları `depo: Depo = Depends(get_depo)` yerine kendi içlerinde doğrudan `from app.depo import depo` ile modül düzeyindeki tek depo nesnesini kullansaydı, bu `dependency_overrides` satırı ne etki yapardı?

(a) Aynı etkiyi yapardı; `dependency_overrides` her durumda rotaların kullandığı depo nesnesini değiştirir.

(b) FastAPI bu durumu otomatik olarak tespit edip bir hata fırlatır, testler hiç çalışmaz.

(c) Hiçbir etkisi olmazdı; rotalar `get_depo` bağımlılığını hiç çağırmadığı için `dependency_overrides` sözlüğüne yapılan bu atama kullanılmaz kalır — testler

Soru 13 (devamı)

modül düzeyindeki gerçek, paylaşılan depoyu kullanmaya devam eder, bir testin oluşturduğu kayıt bir sonraki testte de görünür (veri sızıntısı).

(d) `dependency_overrides`, rotaların `Depends` kullanıp kullanmadığından bağımsız olarak, uygulamadaki tüm modül düzeyi nesneleri otomatik olarak değiştirir.

Soru 14 (8 puan)

4 • Değerlendirme Hedef 4

Aşağıdaki kesitler `app/bagimliliklar.py` ve `tests/conftest.py` dosyalarından alınmıştır.

```python
def get_depo() -> Depo:
    """Rota fonksiyonlarına PAYLAŞILAN tek Depo nesnesini enjekte eder.

    Hf.8'de bu fonksiyonun yerini bir veritabanı oturumu enjekte eden
    get_db_session() alacak; rota fonksiyonlarının bağımlılık ALMA
    biçimi değişmeyecek, yalnızca bağımlılığın NE OLDUĞU değişecek.
    """
    return depo
```

```python
@pytest.fixture
def client():
    test_depo = Depo()
    app.dependency_overrides[get_depo] = lambda: test_depo
    with TestClient(app) as test_client:
        yield test_client
    app.dependency_overrides.clear()
```

(a) `get_depo()` fonksiyonu normal çalışmada (test dışında) hangi nesneyi döndürür — `app/depo.py`'deki `depo = Depo()` satırıyla ilişkisini açıklayın. (b) `client` fixture'ının sonundaki `app.dependency_overrides.clear()` satırı olmasaydı, bu testten sonra çalışacak başka bir testte hangi risk doğardı?

Soru 15 (10 puan)

4 • Değerlendirme Hedef 4

`/openapi.json`'daki `Bilet` şemasının `properties` listesi hiçbir ek kod yazmadan, yalnızca `app/modeller.py`'deki `Bilet` sınıfından otomatik üretilir. Bu otomatik üretimin, Hf.3'ün `interface Bilet` tanımını elle senkron tutmaya çalışmaya göre avantajını, restoran menüsü benzetmesini kullanarak açıklayınız.